Regular Langauge Wrap-UP

Math 252

Regular Langauges

We have been working toward a result that says the following are all equivalent.

1. L can be described by a regular expression.
2. L is recognized by a DFA.
3. L is recognized by an NFA.
4. L is generated by a regular grammar.

The proof that these are equivalent amounts to describing algorithms that can convert from one representation to another. In particular we have shown

a. $1 \Rightarrow 3$
b. $2 \Rightarrow 3$
c. $3 \Rightarrow 2$
d. $2 \Rightarrow 4$
e. $4 \Rightarrow 3$

Still missing from our list is

f. $2 \Rightarrow 1$

Your Mission (You must choose to accept it)

For each conversion a – e, write a brief description of the algorithm. Your description should include the most important "big idea" and also any potential "gotchas", things you have to be careful about or might forget about.

Bonus: Can you figure out how to do conversion f?

A language that is not regular

Since we now have several equivalent ways to show that a langauge is regular, we also have several ways to show that a languages is not regular.

Here is a language that is not regular: $L = \{0^n 1^n \mid n = 0, 1, 2, ...\}$.

1. What are our options for showing the language is not regular?
2. Which do you think will be easiest?
3. See if you can give a convincing argument that L is not regular.
4. What other languages can you construct that are not regular using the same basic idea idea?

Solutions

Compare your descriptions to the descriptions below. Did you leave out anything important? Do the solutions below leave out anything important?

1a. regex → NFA:

- 6 parts:
  - 3 base cases – pretty easy
  - 3 that show union, concatenation and Kleene star – these are the "interesting" part
- union: $A \cup B$ where $A = L(N_1)$ and $B = L(N_2)$.
  - keep all states and transitions from N_1 and N_2
  - add one new start state S; S is final if either start state of N_1 or start state of S_2 is final
  - add some additional transitions
    - if $S_1 \xrightarrow{x} A$, add $S \xrightarrow{x} A$
    - if $S_2 \xrightarrow{x} B$, add $S \xrightarrow{x} B$
- concatenation: AB where $A = L(N_1)$ and $B = L(N_2)$.
  - keep all states and transitions from N_1 and N_2
  - start state from N_1 is the start state
  - only final states from N_2 are final
  - add some additional transitions
    - if $A \xrightarrow{x} F$ (a final state in N_1), then add add $A \xrightarrow{x} S_2$ (start state in N_2)
- Kleene star: $A = L(N_1)$
  - keep all states and transitions from N_1
  - add a new start state that is a final state (because $\lambda \in A^*$)
  - add new transitions from new start state (S)
    - if $S_1 \xrightarrow{x} A$, add $S \xrightarrow{x} A$ (includes case where $A = S_1$)
  - add new transitions to old start state (S_1)
    - if $A \xrightarrow{x} F$, where F is final, add $A \xrightarrow{x} S_1$
    - If we are at the end of a string in $L(N_1)$, this lets us "restart"

1b. DFA → NFA

- easy: a DFA "is" an NFA

1c. NFA → DFA

- DFA states are **subsets of NFA states** (think: magnets on white board)
- A is final in DFA if A contains a final state of NFA
- If there is no transition for a state/symbol combo in NFA, then the transition is to state in DFA

1d. DFA (or NFA) → reg grammar

- $A \xrightarrow{x} B$ becomes
  - $A \to xB$
  - also $A \to x$ if B is final state
- If S is final in DFA, then add $S \to \lambda$

- Number of rules = number of transitions + number of transitions to final states + 1 more if the start state is final

1e. reg grammar $\rightarrow$ NFA

- states: non-terminals of grammar + one additional state (the only final state)
- $S \rightarrow \lambda$ means S is start state
- production rules become transitions in NFA
  - $A \rightarrow xB$ becomes $A \xrightarrow{x} B$
  - $A \rightarrow x$ becomes $A \xrightarrow{x}$ final state